

Universidad de San Carlos de Guatemala
Facultad de Ingeniería
Sistemas Operativos 1
Sección P

Manual Técnico y Guía de Instalación

Carlos Alejandro Rosales Medina
201901657

Guatemala, 20 de agosto de 2026

1. Introducción y Arquitectura General.....	3
1.1 Diagrama de arquitectura.....	3
1.2 Flujo de comunicación entre APIs.....	3
2. Preparación del Entorno (VMs con KVM).....	3
2.1 Instalación de KVM/QEMU y libvirt.....	3
2.2 Imagen base y discos por VM.....	3
2.3 Configuración inicial con cloud-init.....	4
2.4 Creación de las VMs.....	4
2.5 Instalación de runtimes por VM.....	4
VM1 — Containerd.....	4
VM2 — Podman.....	4
VM3 — Docker.....	5
3. Desarrollo y Contenerización de las APIs.....	5
3.1 Estructura del código (ejemplo API1).....	5
3.2 Dockerfile (multi-stage build).....	5
3.3 Construcción de imágenes.....	5
VM1 (Containerd, vía nerdctl).....	6
VM2 (Podman).....	6
3.4 Contenedores en ejecución.....	6
4. Registro Zot y Comunicación Cruzada.....	7
4.1 Instalación de Zot en VM3.....	7
4.2 Publicación de imágenes en Zot.....	7
4.3 Verificación del catálogo.....	8
4.4 Pruebas de comunicación cruzada.....	8
5. Pruebas de Validación Funcional.....	10
5.1 Endpoint /health de cada API.....	10

1. Introducción y Arquitectura General

Este documento describe el diseño, instalación y validación del entorno virtualizado solicitado en el Proyecto 1 del curso Sistemas Operativos 1. El entorno integra 3 máquinas virtuales gestionadas con KVM/libvirt, cada una con un runtime de contenedores distinto (Containerd, Podman y Docker), 3 APIs REST desarrolladas en Go que se comunican entre sí vía HTTP, y un registro privado de imágenes (Zot) donde se centralizan las imágenes construidas en cada VM.

1.1 Diagrama de arquitectura

La siguiente tabla resume la distribución de responsabilidades por máquina virtual:

VM	Hostname	IP	Runtime	Servicio(s)
VM1	vm1-containerd	192.168.122.152	Containerd (+nerdctl)	API1 (puerto 8081), API2 (puerto 8082)
VM2	vm2-podman	192.168.122.87	Podman	API3 (puerto 8083)
VM3	vm3-docker	192.168.122.241	Docker	Registro Zot (puerto 5000)

1.2 Flujo de comunicación entre APIs

Cada API expone un endpoint /health que reporta su propio estado, y dos endpoints call-api# que consultan el /health de otra API para verificar si está operativa. El flujo es el siguiente:

- El cliente solicita GET /api{N}/201901657/call-api{M}
- La API de origen hace un HTTP GET al endpoint /health de la API destino
- Si la respuesta indica status: "UP", se responde con connection: true
- Si hay error o el status no es "UP", se responde con connection: false y un mensaje de ERROR

2. Preparación del Entorno (VMs con KVM)

Host: Ubuntu 24.04 LTS. Se utilizó QEMU/KVM con libvirt para la virtualización.

2.1 Instalación de KVM/QEMU y libvirt

```
sudo apt update
sudo apt install -y qemu-kvm libvirt-daemon-system libvirt-clients \
  bridge-utils virt-manager virtinst genisoimage
sudo systemctl enable --now libvirtd
sudo usermod -aG libvirt,kvm $USER
```

2.2 Imagen base y discos por VM

Se descargó una única imagen base de Ubuntu Server 24.04 (cloud image) y se crearon 3 discos independientes.:

```
wget https://cloud-images.ubuntu.com/releases/24.04/release/\
ubuntu-24.04-server-cloudimg-amd64.img -O ubuntu-base.img

qemu-img create -f qcow2 -F qcow2 -b ubuntu-base.img vm1.qcow2 10G
```

```
qemu-img create -f qcow2 -F qcow2 -b ubuntu-base.img vm2.qcow2 10G
qemu-img create -f qcow2 -F qcow2 -b ubuntu-base.img vm3.qcow2 10G
```

2.3 Configuración inicial con cloud-init

Cada VM recibe su configuración (usuario, contraseña, hostname).

```
# Ejemplo para VM1 (se repite el patrón para VM2 y VM3)
cat > vm1-user-data.yaml << 'EOF'
#cloud-config
hostname: vm1-containerd
users:
  - name: alejandro
    sudo: ALL=(ALL) NOPASSWD:ALL
    groups: sudo
    shell: /bin/bash
    lock_passwd: false
chpasswd:
  list: |
    alejandro:cambiar123
  expire: false
ssh_pwauth: true
EOF

genisoimage -output vm1-cidata.iso -volid cidata -joliet -rock \
-graft-points user-data=vm1-user-data.yaml meta-data=vm1-meta-data.yaml
```

2.4 Creación de las VMs

```
virt-install \
--name vm1-containerd \
--memory 1536 --vcpus 1 \
--disk path=~/.vms/vm1.qcow2,bus=virtio \
--disk path=~/.vms/vm1-cidata.iso,device=cdrom,bus=sata \
--os-variant ubuntu24.04 \
--network network=default,model=virtio \
--graphics none --console pty,target_type=serial \
--import --noautoconsole
```

Nota: se otorgaron permisos de búsqueda (chmod o+x) al directorio home para que el usuario libvirt-qemu pudiera acceder a los discos, ya que por defecto no tiene permisos sobre el home de usuarios distintos a root.

2.5 Instalación de runtimes por VM

VM1 — Containerd

```
sudo apt update
sudo apt install -y containerd
sudo systemctl status containerd
```

VM2 — Podman

```
sudo apt update
sudo apt install -y podman
podman --version
```

VM3 — Docker

```
curl -fsSL https://get.docker.com -o get-docker.sh
sudo sh get-docker.sh
sudo usermod -aG docker $USER
```

Verificación: se confirmó en las 3 VMs mediante `... run hello-world`, que cada runtime descarga y ejecuta contenedores correctamente. Se verificó además que en VM1 y VM2 el comando `docker info` no retorna resultado, confirmando que Docker no está instalado en esas VMs.

3. Desarrollo y Contenerización de las APIs

Las 3 APIs están desarrolladas en Go, cada una como un módulo independiente. Cada una expone su endpoint /health y dos endpoints call-api# hacia las otras dos APIs, siguiendo exactamente el formato de rutas y respuestas JSON.

3.1 Estructura del código (ejemplo API1)

La estructura es equivalente en API2 y API3, cambiando el nombre, VM, puerto y URLs de destino:

```
const (
    apiName = "API1"
    vmName  = "VM1"
    carnet  = "201901657"
    port    = "8081"
)

func healthHandler(w http.ResponseWriter, r *http.Request) {
    resp := HealthResponse{
        Status: "UP", Message: fmt.Sprintf("%s is Ready", apiName),
        Timestamp: time.Now().UTC().Format(time.RFC3339),
        VM: vmName, Carnet: carnet,
    }
    json.NewEncoder(w).Encode(resp)
}
```

3.2 Dockerfile (multi-stage build)

```
FROM golang:1.22-alpine AS build
WORKDIR /app
COPY go.mod ./
COPY main.go ./
RUN go build -o api1 main.go

FROM alpine:latest
WORKDIR /app
COPY --from=build /app/api1 .
EXPOSE 8081
CMD ["/api1"]
```

3.3 Construcción de imágenes

El estándar OCI/Docker exige que el nombre del repositorio de una imagen esté en minúsculas. Por esta razón, las imágenes se nombraron api1-201901657, api2-201901657 y api3-201901657 (equivalente en minúsculas al formato [API#-#CARNET]).

VM1 (Containerd, vía nerdctl)

```
sudo nerdctl build -t api1-201901657:latest .  
sudo nerdctl build -t api2-201901657:latest .
```

VM2 (Podman)

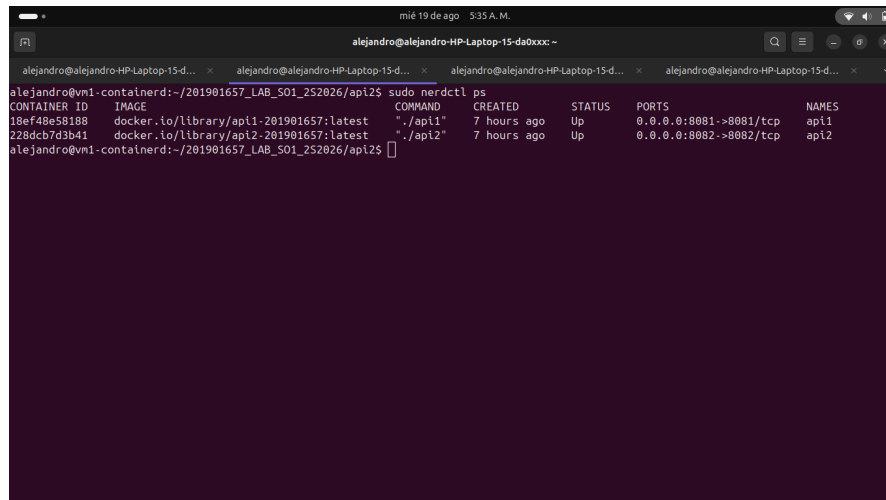
```
podman build -t api3-201901657:latest .
```

3.4 Contenedores en ejecución

VM	Runtime	Comando de ejecución
VM1	nerdctl	sudo nerdctl run -d --name api1 --network so1net -p 8081:8081 api1-201901657:latest
VM1	nerdctl	sudo nerdctl run -d --name api2 --network so1net -p 8082:8082 api2-201901657:latest
VM2	podman	podman run -d --name api3 -p 8083:8083 api3-201901657:latest

Evidencia de los contenedores en ejecución (comando ps de cada runtime):

VM1:



```
mié 19 de ago 5:35 A.M.  
alejandro@alejandro-HP-Laptop-15-d...  
alejandro@alejandro-HP-Laptop-15-d...  
alejandro@vm1-containerd:~/201901657_LAB_S01_252826/api2$ sudo nerdctl ps  
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS                    NAMES  
18ef48e58188   docker.io/library/api1-201901657:latest  "/api1"                 7 hours ago   Up           0.0.0.0:8081->8081/tcp   api1  
228dcb7d3b41   docker.io/library/api2-201901657:latest  "/api2"                 7 hours ago   Up           0.0.0.0:8082->8082/tcp   api2  
alejandro@vm1-containerd:~/201901657_LAB_S01_252826/api2$
```

VM2:

```
mié 19 de ago 5:33 A.M.
alejandro@alejandro-HP-Laptop-15-da0xxx: ~
alejandro@alejandro-HP-Laptop-15-da0xxx: $ virsh console vm2-podman
Connected to domain 'vm2-podman'
Escape character is ^] (Ctrl + )

alejandro@vm2-podman: ~/201901657_LAB_S01_252026/api3$ podman ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS                    NAMES
2e22139aa921   localhost/api3-201901657:latest     ./api3                  7 hours ago   Up 7 hours    0.0.0.0:8083->8083/tcp    api3
alejandro@vm2-podman: ~/201901657_LAB_S01_252026/api3$
```

VM3:

```
mié 19 de ago 5:33 A.M.
alejandro@alejandro-HP-Laptop-15-da0xxx: ~
alejandro@alejandro-HP-Laptop-15-da0xxx: $ virsh console vm3-docker
Connected to domain 'vm3-docker'
Escape character is ^] (Ctrl + )

alejandro@vm3-docker:~$ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS                    NAMES
6a3f7e19dc3c   ghcr.io/project-zot/zot-linux-amd64:latest "/usr/local/bin/zot-..." 3 days ago     Up 7 hours    0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp    zot
alejandro@vm3-docker:~$
```

4. Registro Zot y Comunicación Cruzada

4.1 Instalación de Zot en VM3

Zot se ejecuta como contenedor Docker en VM3, exponiendo el puerto 5000 y con una configuración mínima montada como volumen:

```
docker run -d --name zot -p 5000:5000 \
-v ~/zot/data:/var/lib/registry \
-v ~/zot/config/config.json:/etc/zot/config.json \
ghcr.io/project-zot/zot-linux-amd64:latest
```

```
curl http://localhost:5000/v2/ # 200 OK esperado
```

4.2 Publicación de imágenes en Zot

```
# VM1
sudo nerdctl image convert --oci api1-201901657:latest api1-201901657:oci
sudo nerdctl tag api1-201901657:oci 192.168.122.241:5000/api1-201901657:latest
sudo nerdctl push --insecure-registry 192.168.122.241:5000/api1-201901657:latest

# (mismo procedimiento para api2)

# VM2 — Podman generó un manifest compatible sin necesidad de conversión:
podman tag api3-201901657:latest 192.168.122.241:5000/api3-201901657:latest
podman push 192.168.122.241:5000/api3-201901657:latest
```

En VM2 se agregó además el registro como inseguro:

```
[[registry]]
location = "192.168.122.241:5000"
insecure = true
```

4.3 Verificación del catálogo

```
curl http://192.168.122.241:5000/v2/_catalog | jq .
```

```
mié 19 de ago 3:41 A. M.
alejandro@alejandro-HP-Laptop-15-d...
alejandro@alejandro-HP-Laptop-15-d...
alejandro@vm3-docker:~$ curl -s http://192.168.122.241:5000/v2/_catalog | jq .
{
  "repositories": [
    "api1-201901657",
    "api2-201901657",
    "api3-201901657"
  ]
}
alejandro@vm3-docker:~$
```

4.4 Pruebas de comunicación cruzada

Se validaron las 6 combinaciones posibles de comunicación entre las 3 APIs, todas con resultado connection: true:

Origen	Destino	Comando	Resultado
API1 (VM1)	API2 (VM1)	curl localhost:8081/api1/201901657/call-api2	connection: true
API2 (VM1)	API1 (VM1)	curl localhost:8082/api2/201901657/call-api1	connection: true
API1 (VM1)	API3 (VM2)	curl localhost:8081/api1/201901657/call-api3	connection: true

API2 (VM1)	API3 (VM2)	curl localhost:8082/api2/201901657/call-api3	connection: true
API3 (VM2)	API1 (VM1)	curl localhost:8083/api3/201901657/call-api1	connection: true
API3 (VM2)	API2 (VM1)	curl localhost:8083/api3/201901657/call-api2	connection: true

Evidencia de las 6 pruebas de comunicación cruzada (una captura por cada llamada, mostrando connection: true):

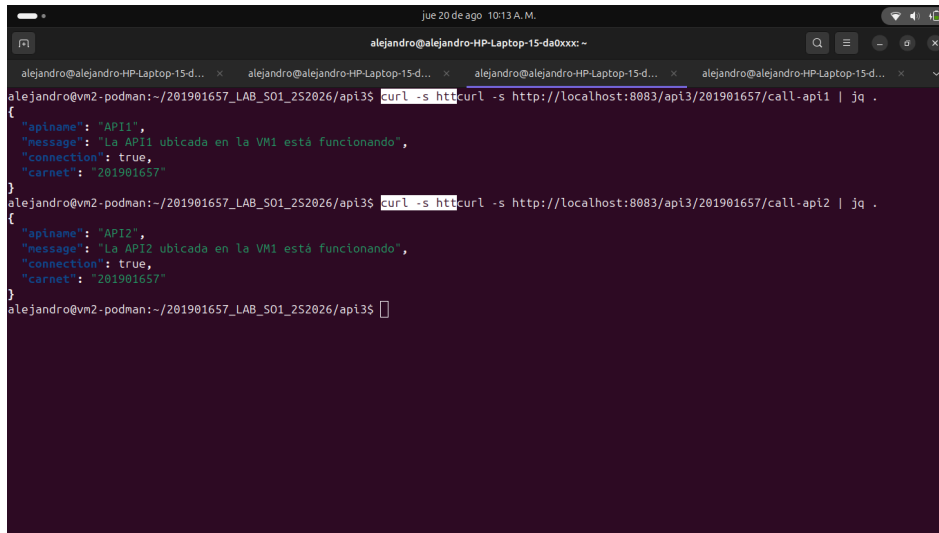
VM1:

```

jue 20 de ago 10:12 A.M.
alejandro@alejandro-HP-Laptop-15-da0xxx -
alejandro@alejandro-HP-Laptop-15-d... x alejandro@alejandro-HP-Laptop-15-d... x alejandro@alejandro-HP-Laptop-15-d... x alejandro@alejandro-HP-Laptop-15-d... x
alejandro@vm1-containerd:~/201901657_LAB_S01_252026/api2$ curl -s http://localhost:8081/api1/201901657/call-api2 | jq .
{
  "apiname": "API2",
  "message": "La API2 ubicada en la VM1 est\u00e1 funcionando",
  "connection": true,
  "carnet": "201901657"
}
curl -s http://localhost:8081/api1/201901657/call-api3 | jq .
{
  "apiname": "API3",
  "message": "La API3 ubicada en la VM2 est\u00e1 funcionando",
  "connection": true,
  "carnet": "201901657"
}
curl -s http://localhost:8082/api2/201901657/call-api1 | jq .
{
  "apiname": "API1",
  "message": "La API1 ubicada en la VM1 est\u00e1 funcionando",
  "connection": true,
  "carnet": "201901657"
}
curl -s http://localhost:8082/api2/201901657/call-api3 | jq .
{
  "apiname": "API3",
  "message": "La API3 ubicada en la VM2 est\u00e1 funcionando",
  "connection": true,
  "carnet": "201901657"
}

```

VM2:



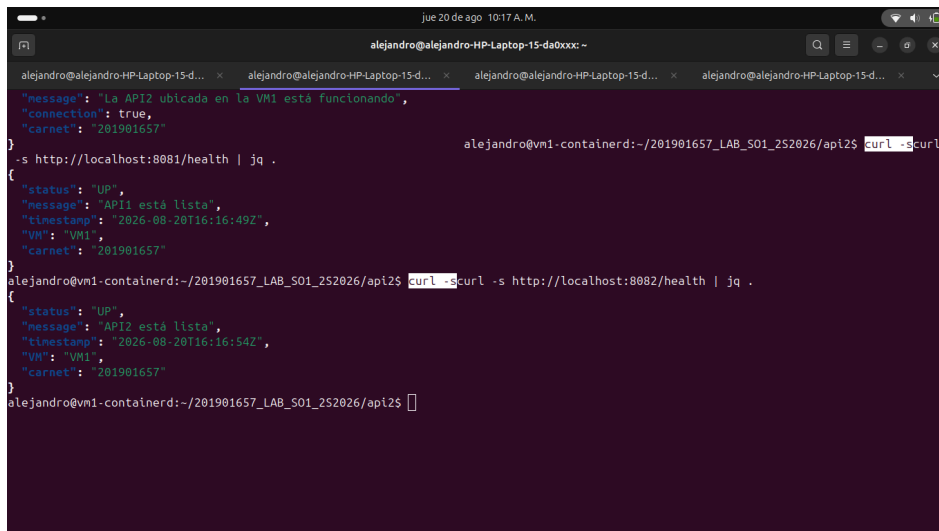
```
jue 20 de ago 10:13 A. M.
alejandro@alejandro-HP-Laptop-15-da0xxx: ~
alejandro@alejandro-HP-Laptop-15-d... x alejandro@alejandro-HP-Laptop-15-d... x alejandro@alejandro-HP-Laptop-15-d... x alejandro@alejandro-HP-Laptop-15-d... x
alejandro@vm2-podman: ~/201901657_LAB_S01_252026/api3$ curl -s http://localhost:8083/api/201901657/call-api1 | jq .
{
  "apiName": "API1",
  "message": "La API1 ubicada en la VM1 está funcionando",
  "connection": true,
  "carnet": "201901657"
}
alejandro@vm2-podman: ~/201901657_LAB_S01_252026/api3$ curl -s http://localhost:8083/api/201901657/call-api2 | jq .
{
  "apiName": "API2",
  "message": "La API2 ubicada en la VM1 está funcionando",
  "connection": true,
  "carnet": "201901657"
}
alejandro@vm2-podman: ~/201901657_LAB_S01_252026/api3$
```

5. Pruebas de Validación Funcional

5.1 Endpoint /health de cada API

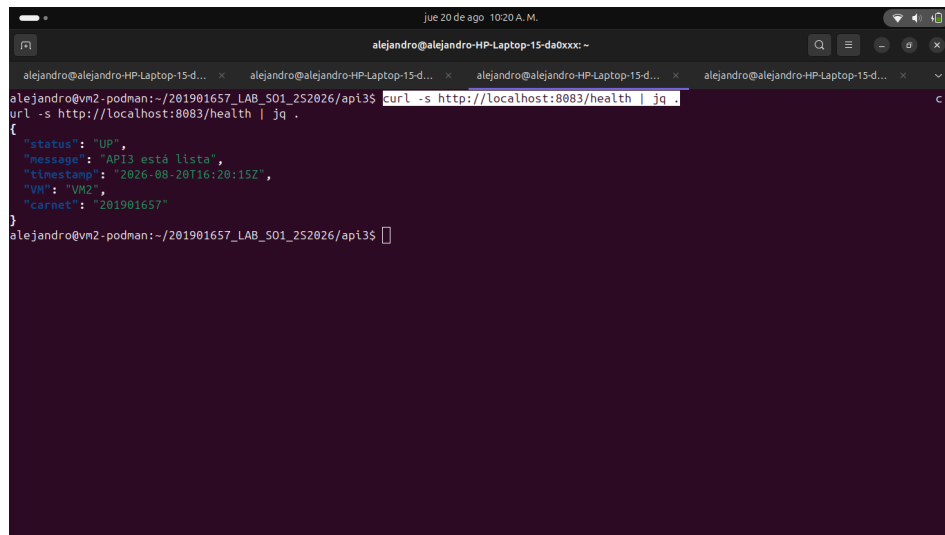
```
curl -s http://localhost:8081/health | jq .
curl -s http://localhost:8082/health | jq .
curl -s http://localhost:8083/health | jq .
```

VM1:



```
jue 20 de ago 10:17 A. M.
alejandro@alejandro-HP-Laptop-15-da0xxx: ~
alejandro@alejandro-HP-Laptop-15-d... x alejandro@alejandro-HP-Laptop-15-d... x alejandro@alejandro-HP-Laptop-15-d... x alejandro@alejandro-HP-Laptop-15-d... x
"message": "La API2 ubicada en la VM1 está funcionando",
"connection": true,
"carnet": "201901657"
}
alejandro@vm1-containerd: ~/201901657_LAB_S01_252026/api2$ curl -s http://localhost:8081/health | jq .
{
  "status": "UP",
  "message": "API1 está lista",
  "timestamp": "2026-08-20T16:16:49Z",
  "VM": "VM1",
  "carnet": "201901657"
}
alejandro@vm1-containerd: ~/201901657_LAB_S01_252026/api2$ curl -s http://localhost:8082/health | jq .
{
  "status": "UP",
  "message": "API2 está lista",
  "timestamp": "2026-08-20T16:16:54Z",
  "VM": "VM1",
  "carnet": "201901657"
}
alejandro@vm1-containerd: ~/201901657_LAB_S01_252026/api2$
```

VM2:



```
jue 20 de ago 10:20 A. M.
alejandro@alejandro-HP-Laptop-15-da0xxx: -
alejandro@alejandro-HP-Laptop-15-d... x alejandro@alejandro-HP-Laptop-15-d... x alejandro@alejandro-HP-Laptop-15-d... x alejandro@alejandro-HP-Laptop-15-d... x
alejandro@vm2-podman:~/201901657_LAB_S01_252026/api3$ curl -s http://localhost:8083/health | jq .
{
  "status": "UP",
  "message": "API3 está lista",
  "timestamp": "2026-08-20T16:20:15Z",
  "VM": "VM2",
  "carnet": "201901657"
}
alejandro@vm2-podman:~/201901657_LAB_S01_252026/api3$
```